Dynamic GraphRAG for Open Scientific QA

Concepts, Design Decisions, and Rationale

Project Team

April 13, 2026

Contents

1 Executive Summary **3**

2 The Core Idea: Why a Graph? **3**
2.1 The Limitations of Flat Retrieval . . . 3
2.2 What the Graph Layer Adds . . . 3
2.3 Why Dynamic, Not Static? . . . 4

3 Full Pipeline Architecture **4**
3.1 Stage 1: Query Planning . . . 4
3.2 Stage 2: Domain Detection . . . 5
3.3 Stage 3: Query Expansion + Dual Retrieval . . . 5
3.4 Stage 4: Adaptive Paper Filtering . . . 6
3.5 Stage 5: Entity and Relation Extraction . . . 7
3.6 Stage 6: Graph Construction with Entity Resolution . . . 8
3.7 Stage 7: Subgraph Retrieval with Hub Penalty . . . 9
3.8 Stage 8: Confidence-Driven Adaptive Expansion . . . 10
3.9 Stage 8b: Intent-Driven Entity Weighting and Strict Relevance Gating . . . 11
3.10 Stage 9: Answer Generation with Intent Instruction . . . 11
3.11 Stage 10: Answer Compilation and Coverage Validation . . . 13
3.12 Confidence Scoring and Adaptive Thresholds . . . 16

4 Cross-Cutting Design Decisions **17**
4.1 Domain Agnosticity . . . 17
4.2 Session-Aware Caching and Incremental State Management . . . 17
4.3 Failure Handling Philosophy . . . 19
4.4 Adaptive Answer Context Policy . . . 19

5 Evaluation Framework **19**
5.1 Benchmark Design . . . 19
5.2 Ablation Modes . . . 20
5.3 Metrics . . . 20

6 Key Improvements Summary **20**

7 Operational Behavior Under Different Conditions **21**

8 Component Map **22**

9 Tradeoffs and Known Constraints **22**

10 Summary Narrative **23**

11 Next Steps **23**

1 Executive Summary

This project implements a **Dynamic GraphRAG** system for open scientific question answering. The fundamental problem it addresses is that classical Retrieval-Augmented Generation (RAG) treats retrieved documents as flat, unstructured context—it concatenates the top-k chunks and hands them to a language model. This works for simple factual questions, but fails when the answer requires *connecting concepts across multiple papers*, understanding *causal chains*, or *comparing methods* that no single document describes side by side.

Our system solves this by introducing an intermediate **knowledge graph layer**: for each user query, it retrieves fresh scientific papers, extracts entities and relations, builds a temporary query-scoped graph, retrieves a focused subgraph with multi-hop reasoning paths, and then compiles a structured, evidence-grounded answer.

The three objectives we optimized for simultaneously:

- **Answer quality**: structured, multi-hop, evidence-backed outputs that go beyond single-document summarization.
- **Robustness**: graceful degradation under API failures, rate limits, and malformed LLM outputs—the system never crashes, it always returns *something* useful.
- **Performance**: targeted optimizations (parallelism, caching, session reuse) that reduce latency without sacrificing correctness.

2 The Core Idea: Why a Graph?

2.1 The Limitations of Flat Retrieval

Classic RAG follows a simple pipeline: query → retrieve top-k documents → concatenate into context → generate answer. This approach has three structural weaknesses for scientific QA:

1. **No cross-document reasoning.** If the causal chain "drought → soil moisture loss → crop stress → yield reduction" is spread across three papers, flat retrieval gives the LLM three disconnected chunks. The model must *implicitly* reconstruct the chain, which it often fails to do reliably.
2. **No structural signal.** All retrieved chunks look the same to the model—just paragraphs of text. There is no way to distinguish a central concept from a peripheral mention, or a causal relationship from a correlational one.
3. **Context dilution.** With 5–15 paper abstracts in the prompt, the LLM's attention spreads thin. Important evidence competes with boilerplate methodology descriptions and unrelated findings.

2.2 What the Graph Layer Adds

A knowledge graph provides three things flat text cannot:

- **Explicit structure**: entities become nodes, relationships become edges. The model receives "(drought) $\xrightarrow{\text{causes}}$ (soil moisture loss) $\xrightarrow{\text{leads_to}}$ (crop stress)" rather than hoping it will parse this from paragraphs.

- **Multi-hop paths**: the graph retriever can traverse 2–4 hop reasoning chains that span multiple papers, surfacing connections no single document contains.
- **Focused context**: instead of dumping entire abstracts, we send only the relevant subgraph—the entities and relations that actually matter for this query.

2.3 Why Dynamic, Not Static?

We considered maintaining a persistent global knowledge graph (built offline from a corpus), but rejected it for three reasons:

1. **Staleness.** Scientific knowledge evolves rapidly. A graph built last week may miss the latest findings.
2. **Contamination.** A global graph contains entities and relations from many domains and topics. For a specific query about lithium extraction methods, the presence of thousands of unrelated nodes adds noise to retrieval.
3. **Scalability.** Maintaining and updating a global graph requires significant infrastructure. A per-query ephemeral graph is simpler and self-contained.

Our approach builds a **temporary in-memory graph per query**, scoped exactly to the retrieved papers. This keeps retrieval aligned with the user's current intent, avoids stale knowledge contamination, and requires zero persistent storage.

3 Full Pipeline Architecture

The system executes a 10-stage pipeline for each query. Each stage was designed to solve a specific failure mode observed in earlier iterations.

$$\text{Query} \rightarrow \text{Planning} \rightarrow \text{Domain Routing} \rightarrow \text{Dual Retrieval} \rightarrow \text{Adaptive Filtering}$$
$$\rightarrow \text{Entity Extraction} \rightarrow \text{Graph Construction} \rightarrow \text{Subgraph Retrieval}$$
$$\rightarrow \text{Confidence Check} \xrightarrow{\text{if low}} \text{Adaptive Retry}$$
$$\rightarrow \text{Answer Generation} + \text{Compilation} \rightarrow \text{Final Output}$$

3.1 Stage 1: Query Planning

What it does. Before any retrieval happens, the query planner classifies the question's *reasoning type* and extracts structural requirements. It produces a `QueryPlan` containing:

- **Reasoning type**: one of `definition`, `comparison`, `method_process`, `causal_mechanism`, or `optimization`.
- **Key entities**: the core concepts the user is asking about.
- **Dimensions**: analysis angles (e.g., "effectiveness", "mechanism", "cost").
- **Intent signals**: 1–2 phrases capturing the true user need.
- **Complexity**: simple / moderate / complex (controls how large the graph needs to be).

- **Graph requirements**: minimum nodes, edges, and entity types needed for a satisfactory answer.

Why this matters. Every downstream stage uses the plan. A comparison question needs different entity types than a causal question. A complex multi-part question requires a denser graph than a simple definition query. Without planning, the system would treat all queries identically—retrieving the same number of papers, extracting the same entity types, and structuring the answer the same way regardless of what was asked.

Why LLM + heuristic fallback. The planner first attempts zero-shot classification via the Groq LLM. If the LLM is unavailable (rate limit, timeout), it falls back to keyword-based heuristic classification with fuzzy matching. This dual approach ensures planning never fails: the LLM gives nuanced classification; the heuristics provide guaranteed availability.

Why not just keywords? Pure keyword classification mishandles ambiguous queries. "What are methods for reducing crop loss?" contains "methods" (suggesting `method_process`) but also "reducing" (suggesting `optimization`). The LLM can weigh context; keywords cannot. Conversely, the LLM sometimes returns unexpected types or fails under load, so heuristics provide a safety net.

3.2 Stage 2: Domain Detection

What it does. Classifies the query into one of five scientific domains (Agriculture, Geoscience, Computer Science, Supply Chain, Environment) to guide retrieval source selection and filtering.

Three-tier strategy.

1. **LLM classification** (Groq): zero-shot prompt asking for domain.
2. **Keyword matching**: domain-specific keyword dictionaries as fallback.
3. **Embedding similarity**: cosine similarity between query embedding and domain reference sentences.

Why three tiers? Each method has different failure modes. The LLM gives the best results but can be unavailable. Keywords are fast and deterministic but miss novel terminology. Embeddings handle semantic similarity but can confuse related domains. The cascade ensures classification *always* succeeds.

Why not skip domain detection? We could retrieve from all sources for every query, but this wastes API calls and returns irrelevant papers. A geoscience query sent to a biomedical API returns noise. Domain routing improves both precision and latency.

3.3 Stage 3: Query Expansion + Dual Retrieval

What it does. The paper retriever performs two key operations:

Query Expansion. An LLM expands the user's question into a denser retrieval query by adding synonyms, related terms, and implicit subtopics. For example, "What causes coral bleaching?" might expand to include "thermal stress", "symbiodinium expulsion", "ocean acidification", and "sea surface temperature anomalies".

Dual Retrieval. Papers are retrieved for *both* the original query and the expanded query from multiple academic APIs (Europe PMC, arXiv, OpenAlex), then deduplicated by title.

Why expansion matters. Scientific terminology mismatch is a major failure mode. A user asking about "crop disease resistance" may not use the terms that papers use ("pathogen defense mechanisms", "R-gene mediated immunity"). Expansion bridges this vocabulary gap.

Why validate the expansion. Uncontrolled expansion can drift far from the original intent. We enforce that the expanded query must retain ≥50% of the original query's anchor terms. Without this, an expansion of "lithium extraction methods" could drift toward "battery recycling economics"—topically related but not what was asked.

Why dual retrieval instead of just expanded? The original query captures the user's precise intent; the expanded query captures broader coverage. Running both and deduplicating gives us precision (original) + recall (expanded). Using only the expanded query risks losing the user's exact focus.

Why multiple APIs? No single scholarly API has complete coverage. Europe PMC excels at biomedical/agricultural literature. arXiv covers recent computer science and physics preprints. OpenAlex provides broad fallback coverage. The system tries sources in order and falls back gracefully when any source returns too few results (<3 papers).

Plan-Aware Scoring. Retrieved papers are scored using a weighted combination:

$$\text{score} = 0.6 \cdot \text{semantic_similarity} + 0.25 \cdot \text{keyword_coverage} + 0.15 \cdot \text{metadata_prior}$$

where semantic similarity uses cosine distance between embeddings, keyword coverage measures overlap with query terms, and the metadata prior rewards citation count and recency. This weighting was chosen because semantic similarity is the strongest relevance signal, but keyword overlap catches cases where embeddings miss exact technical terms, and metadata helps prefer well-cited, recent work.

3.4 Stage 4: Adaptive Paper Filtering

What it does. Filters the retrieved paper pool to keep only the most relevant candidates. The filtering pipeline:

1. **Abstract filter**: drop papers without usable abstracts (<50 characters).
2. **Date filter**: drop papers older than a configurable minimum year.
3. **Semantic ranking**: embed all paper titles/abstracts and score by cosine similarity to the query.
4. **Adaptive percentile filter**: keep papers in the top 60th percentile by similarity.
5. **Intent-aware boosting**: adjust scores based on query intent (e.g., "solution" intent boosts papers mentioning "improve", "optimize"; "causal" intent boosts papers mentioning "mechanism", "driver").
6. **Top-k truncation**: keep the highest-scoring papers up to the configured limit.

Why adaptive percentile instead of a fixed threshold? We initially used a fixed similarity threshold (e.g., 0.4), but this is brittle. For a well-studied topic, most papers score above 0.4 and the threshold does nothing. For a niche topic, even the best papers score below 0.4 and everything

gets filtered out. Percentile-based filtering adapts to the score distribution of each query: it always keeps the relatively best papers regardless of absolute scores.

Why intent-aware boosting? Not all relevant papers are equally useful for a given question type. For a "What methods exist for X?" question, a paper describing a novel technique is more useful than a paper surveying the history of X, even if both are semantically similar. Intent-aware boosting promotes papers whose content aligns with what the user actually needs.

3.5 Stage 5: Entity and Relation Extraction

What it does. Extracts structured entities and relations from each paper's abstract using LLM prompts. Entities are classified into six domain-agnostic types:

- **Concept / Entity**: organisms, materials, systems, datasets, models, genes.
- **Problem / Condition**: diseases, failures, risks, bugs, anomalies.
- **Method / Intervention**: techniques, algorithms, treatments, protocols, tools.
- **Context / Location**: environments, regions, platforms, settings.
- **Cause / Factor**: drivers, risk factors, inputs, constraints, stressors.
- **Effect / Outcome**: results, metrics, impacts, performance, consequences.

Relations use ten types: `affects`, `mitigates`, `occurs_in`, `caused_by`, `leads_to`, `detected_by`, `studied_in`, `correlates_with`, `depends_on`, `part_of`.

Why domain-agnostic types? We considered creating domain-specific type systems (e.g., Gene, Protein, Pathway for biomedical; Layer, Formation, Basin for geoscience), but this would require maintaining separate vocabularies per domain, and new domains would need new types. The six abstract categories map naturally to any scientific domain: a gene and a geological formation are both "Concept / Entity"; a treatment and an algorithm are both "Method / Intervention". This keeps the system extensible without per-domain engineering.

Why question-focused extraction. The extraction prompt includes the user's question and the query plan's entities and dimensions. This focuses the LLM on extracting entities *relevant to the query*, rather than exhaustively extracting every entity in the abstract. Without this, a paper about "climate change impacts on agriculture, economic models, and policy frameworks" would produce dozens of entities when only the agriculture-related ones matter for a crop science question.

Why parallel extraction with bounded concurrency. Extraction is the most latency-heavy stage because it requires one LLM call per paper. Running these calls in parallel (4 concurrent workers) reduces wall-clock time from ∼40s (sequential, 8 papers) to ∼10s. We bound concurrency to prevent overwhelming the API provider and triggering rate limits.

Why multi-level retry. LLM extraction frequently produces malformed JSON, especially under load. Our retry cascade:

1. Retry with exponential backoff (0.5s → 1.0s + jitter).
2. Re-prompt with strict JSON formatting instructions.
3. Fall back to alternative model if primary is unavailable.

4. Fall back to deterministic heuristic extraction (regex-based entity and relation extraction from text) if all LLM calls fail.

This cascade means extraction *never* returns empty-handed. Even under complete API failure, the heuristic fallback produces a minimal but usable set of entities and relations, preserving pipeline continuity.

Extraction caching. Results are cached by paper ID / abstract hash. This avoids re-extracting the same paper when the adaptive retry loop re-runs extraction or when a follow-up query uses overlapping papers.

3.6 Stage 6: Graph Construction with Entity Resolution

What it does. Builds a directed knowledge graph (`nx.DiGraph`) from extracted entities and relations, then optimizes it for the query.

Entity Resolution. Raw extraction almost always produces near-duplicate nodes: "machine learning" and "ML", "Bacillus thuringiensis" and "Bt", "CRISPR-Cas9" and "CRISPR". Without resolution, these become separate nodes with separate edges, fragmenting the graph and weakening reasoning paths. Entity resolution merges duplicates using:

- Canonicalization (lowercasing, whitespace normalization).
- Lexical signature matching (sorted character n-grams).
- Abbreviation detection ("ML" $\rightarrow$ "machine learning").
- Token similarity and containment ("deep learning model" contains "deep learning").
- Embedding clustering for semantically identical but lexically different names.

Why this multi-signal approach? No single signal catches all duplicates. Lexical matching catches "Machine Learning" / "machine learning" but misses "ML". Abbreviation detection catches "ML" / "Machine Learning" but misses "deep learning" / "DL-based approach". Embedding similarity catches semantic equivalence but produces false positives for related-but-distinct concepts ("supervised learning" and "unsupervised learning" embed closely). Combining signals with conservative thresholds gives high-precision merging.

Graph Optimization. After initial construction, the graph is optimized through four operations:

1. **Generic node removal** (only on graphs >12 nodes): removes overly broad nodes like "research", "study", "system" that add no informational value and can dominate centrality metrics.
2. **Co-occurrence edge addition**: if two entities appear in the same paper, they likely share a meaningful relationship even if extraction did not produce an explicit edge. We add edges between co-occurring entities (threshold: $\geq$2 co-occurrences) to densify sparse graphs.
3. **Semantic bridge edges**: for disconnected or weakly connected components, we compute pairwise embedding similarity between non-adjacent nodes and add edges where similarity exceeds 0.62. This enables multi-hop paths between concepts from different papers that are semantically related but never co-occurred. Limited to top 2 bridges per node to avoid over-densification.

4. **Isolated node pruning** (only on graphs >12 nodes): removes degree-0 nodes unless they are semantically close to the query. This cleans up extraction artifacts without losing query-relevant signal.

Why these thresholds? The 12-node minimum for pruning prevents over-aggressive cleanup on small graphs where every node matters. The 0.62 semantic bridge threshold was tuned to allow genuine cross-paper connections ("soil erosion" ↔ "land degradation") while rejecting spurious ones ("soil erosion" ↔ "erosion of public trust"). The 2-co-occurrence threshold ensures edges represent genuine topical association, not coincidental mention.

Why both co-occurrence and semantic bridges? Co-occurrence edges are high-precision (same paper = likely related) but only connect entities within the same document. Semantic bridges are lower-precision but connect entities across documents—which is exactly where multi-hop reasoning needs to work.

3.7 Stage 7: Subgraph Retrieval with Hub Penalty

What it does. Extracts a query-relevant subgraph from the full graph, along with ranked reasoning paths.

Seed Selection. Embed all node labels and the user question, compute cosine similarity, and select the top-5 most relevant nodes as seeds (threshold: similarity > 0.35). If no node clears the threshold, fall back to the top-5 by ranked similarity. Seeds anchor the subgraph to the user's question.

BFS Expansion. Starting from seeds, traverse the graph up to 2 hops, collecting all reachable nodes and edges. This captures the local neighborhood around query-relevant entities.

Hub Penalty. High-degree "hub" nodes (e.g., "climate change" in an environmental graph) connect to many other nodes. Without a penalty, these hubs dominate the subgraph, pulling in generic connections and diluting specific evidence. The hub penalty downweights nodes proportional to their degree, promoting specific, informative entities over generic connectors.

Why hub penalty instead of just removing hubs? Hubs are not useless—they represent important concepts. The penalty reduces their *dominance* without eliminating them entirely. A query about "how does climate change affect coral reefs" should include "climate change" as a node, but the subgraph should be centered on the coral-specific connections, not on everything climate change connects to.

Reasoning Path Extraction. The retriever finds all simple paths of length 2–4 between subgraph nodes, ranks them by quality (entity type diversity, relation type priority), and keeps the top 5. These paths become explicit reasoning chains in the answer context:

$$\text{(thermal stress)} \xrightarrow{\text{causes}} \text{(symbiont expulsion)} \xrightarrow{\text{leads_to}} \text{(coral bleaching)}$$

Why explicit paths? Without explicit paths, the LLM must reconstruct reasoning chains from a flat list of triples. With explicit paths, the reasoning structure is handed to the model ready-made. This significantly improves the quality of multi-hop answers, especially for causal and process questions.

Graph Confidence Scoring. The retriever computes a confidence score:

$$\text{confidence} = 0.25 \cdot \text{density} + 0.25 \cdot \text{degree_norm} + 0.20 \cdot \text{edge_norm} + 0.30 \cdot \text{seed_norm}$$

This score drives the adaptive expansion loop: if confidence is below 0.55, the system triggers additional retrieval.

3.8 Stage 8: Confidence-Driven Adaptive Expansion

What it does. When graph confidence is below the threshold (0.55), the system does not simply accept a weak answer. Instead, it runs a bounded retry loop:

1. Analyze what the graph is missing (from coverage validation: "need $\geq$3 Method nodes, have 1").
2. Generate a targeted expansion query focusing on the missing types (e.g., "What are alternative techniques and procedures for X?").
3. Retrieve additional papers with the expanded query.
4. Re-extract entities from new papers.
5. Merge the new entities/relations into the existing graph (deduplicating by node ID).
6. Re-run subgraph retrieval and confidence scoring.
7. If still insufficient and iteration < 2, repeat.

Why this is the key differentiator. Most RAG systems retrieve once and answer once. If the initial retrieval was poor, the answer is poor—end of story. Our system *measures* answer quality through graph confidence and coverage validation, *diagnoses* what is missing, and *acts* to fill the gap. This is analogous to how a human researcher would respond to finding insufficient evidence: search again with different terms, focusing on what is missing.

Why cap at 2 iterations? Each expansion adds 10–20 seconds of latency (retrieval + extraction + graph building). More than 2 iterations gives diminishing returns: if the literature does not contain enough evidence after two targeted expansions, a third is unlikely to help. The cap prevents runaway latency.

Why targeted expansion instead of generic re-retrieval? Re-running the same query returns the same papers. The adaptive retry analyzes *what specific evidence is missing* and generates queries that target those gaps. If the graph lacks "Method/Intervention" nodes, the expansion query explicitly asks for techniques and procedures. This is far more effective than broadening the search indiscriminately.

Safeguards Against Expansion Drift.

- Expansion confidence checks consider validation state to avoid unnecessary retries on already-valid subgraphs.
- Repeated refinement queries are detected and stopped early to prevent looped retrieval drift.
- Refinement hints emphasize entities/dimensions/intent only; missing-requirement labels are no longer injected into retrieval text (this caused retrieval queries like "Need 3 Method nodes for lithium extraction" which confuse scholarly APIs).

3.9 Stage 8b: Intent-Driven Entity Weighting and Strict Relevance Gating

What it does. After the initial graph is built and before LLM generation, two optional screening layers are applied to sharpen the subgraph toward query intent:

Intent-Driven Entity Weighting. Entities in the subgraph are reweighted based on the query plan's reasoning type:

- **method_process intent**: boost Method/Intervention and Context nodes; demote generic Concept nodes.
- **causal_mechanism intent**: boost Cause/Factor and Effect/Outcome nodes; demote Context nodes.
- **comparison intent**: boost Entity and Effect/Outcome nodes; demote standalone Method nodes.
- **definition intent**: boost Concept/Entity and Property nodes; demote Problem/Condition nodes.

These weights are applied during subgraph retrieval: edges involving high-weight node types are prioritized, and node selection algorithms (seed expansion, path ranking) favor aligned entity types. This ensures the graph retriever prioritizes entities and connections aligned with what the user actually asked for, not generic entities that happen to be well-connected.

Strict Relevance Gate (Optional). For complex or high-noise domains, a second-pass relevance filter can optionally be applied:

- Score each subgraph entity by semantic similarity to the query (embeddings).
- Combine semantic similarity (70% weight) with lexical overlap to the query (30% weight).
- Remove entities scoring below the adaptive percentile threshold (default: 20th percentile).

This gate is particularly valuable for broad domains (Computer Science, Chemistry) where papers often cover multiple subtopics and the initial graph can acquire off-topic entities from co-occurrence or semantic bridge edges. The gate must be configured per-deployment because overly aggressive pruning on narrow, well-focused domains can remove valid evidence.

When activated: Strict relevance gating is disabled by default. It activates only when:

- The query plan complexity is marked as "complex" AND
- The subgraph has >12 nodes (enough to afford pruning without losing all structure) AND
- Deployment configuration explicitly enables '`strict_mode`' for this query.

3.10 Stage 9: Answer Generation with Intent Instruction

What it does. Generates a natural-language answer using the Groq LLM, conditioned on graph context and the query plan's reasoning type.

Context Construction. The LLM prompt assembles a multi-part context:

- Graph triples: "(entity_a) $\xrightarrow{\text{relation}}$ (entity_b)"—up to 30 triples, pre-filtered by relevance.
- Reasoning paths: explicit 2–4-hop chains ranked by quality and diversity.
- Paper abstracts: the top-3 highest-cited or best-matched papers, included when the graph is sparse (<3 edges) to ensure narrative depth.
- Intent instruction: reasoning-type-specific guidance (see below).
- Confidence instruction: included when graph confidence is below 0.55; instructs cautious language.
- Citation format: consistent [Paper: title] reference format.

Why intent instruction? The LLM alone cannot reliably adjust answer format and comprehensiveness based on reasoning type. Intent instructions make the expectation explicit: "This is a comparison question, provide 4–6 aspects with specific trade-offs" vs. "This is a definition question, use 2–3 sentences for the main idea and list 3 related concepts". This results in significantly better answer structure without post-processing.

Intent-Driven Answer Templates. The answer structure varies by reasoning type, and the intent instruction specifies both format and minimum item count:

1. **method_process** (min 6 items): ordered list of techniques with step dependencies. Instruction: "Provide step-by-step procedures, indicate which steps are prerequisites for later ones, and include any constraints or alternatives".
2. **comparison** (min 4 items): aspect-vs.-alternative table or matrix with metrics. Instruction: "Identify 4+ comparison aspects, show trade-offs (what improves and what worsens across alternatives), and highlight where consensus exists vs where approaches diverge".
3. **causal_mechanism** (min 4 items): linear or branching chain "Cause → Intermediate mechanism(s) → Effect". Instruction: "Separate root causes from triggering factors; show conditional branches (e.g., 'mechanism A occurs if X is high, mechanism B occurs if X is low')".
4. **definition** (variable, typically 3–5 elements): main definition + related concepts + constraints. Instruction: "Define the term concisely in 2 sentences; distinguish it from related terms; note context where the definition varies".
5. **optimization** (min 5 items): trade-off matrix and recommendation. Instruction: "Present 5+ alternatives with their trade-offs; identify the recommended option(s) for different scenarios".
6. **default** (min 4 items): key findings list. Instruction: "Summarize 4+ main findings from the retrieved papers with specific metrics where available".

Why variable vs. fixed templates? Different reasoning types have different information density requirements. A definition should be concise; a method list should be comprehensive. Using different templates per type avoids both underfitting (too-short answers to complex questions) and padding (lengthy answers to simple questions).

Sparse Graph Supplementation. When the extracted graph has <3 nodes or <2 edges, top-3 paper abstracts or full paragraphs are appended directly to the LLM prompt. This ensures the model always has enough narrative depth: even if structured entity/relation extraction was sparse, the raw text provides context for the LLM to ground its answer.

Low Confidence Mode. When graph confidence is below threshold (0.55), an additional confidence instruction is prepended:

> "NOTE: Confidence in this answer is low (limited supporting evidence). Use cautious language: 'may', 'suggests', 'some evidence indicates' rather than definitive claims. Separately flag where evidence is mixed, conflicting, or sparse. Indicate which claims are well-supported vs which are inferred."

This instructs the LLM to use hedging language and explicitly separate strong vs weak evidence, improving answer transparency even when the evidence base is weak.

Rate-Limited Fallback Answer. If Groq API returns rate-limit (429) errors even after backoff retries (exponential up to 8 seconds), the system generates a deterministic fallback answer directly from retrieved papers:

- Extract the first 1–2 sentences from each paper's abstract.
- Annotate each with paper metadata: [Paper: title (year, first author)].
- Optionally include the top 2–3 reasoning paths if the graph is locally available.
- Prepend a header: "LLM generation was rate-limited. The following summary is assembled directly from retrieved papers."

This ensures users always receive *something useful* even during API stress, avoiding blank answers or error messages. While less polished than LLM-generated text, the fallback is grounded, traceable, and immediate.

3.11 Stage 10: Answer Compilation and Coverage Validation

What it does. The answer compiler is the final quality gate. It operates directly on the graph structure—not on LLM-generated text—to produce a structured answer with provenance.

Node Ranking and Selection Algorithm.

$$\text{score} = 0.4 \cdot \text{degree_centrality} + 0.3 \cdot \text{closeness_to_seeds} + 0.3 \cdot \text{paper_support}$$

- **Degree centrality** (40% weight): nodes with many incoming/outgoing edges are structurally important—they serve as connectors or hubs. An unusually high weight because structural importance often correlates with conceptual importance.
- **Closeness to seeds** (30% weight): seeds are the top-5 nodes most similar to the query. Nodes close to seeds (under 2 hops) are query-relevant. This ensures selected nodes answer the *specific question*, not just general information about the domain.
- **Paper support** (30% weight): how many distinct papers cite or create this node. Single-paper nodes are less robust than multiauthored findings. Nodes with support from 3+ papers rank significantly higher.

This three-signal ranking avoids over-relying on any single indicator. A query-irrelevant hub (high centrality but low seed closeness) is ranked lower than a query-specific node with lower centrality but stronger paper backing.

Reasoning-Type-Aware Node Selection. After scoring all nodes, the compiler selects node *types* based on the reasoning type:

- `method_process`: prioritize Method/Intervention nodes (primary), then Effect/Outcome and Context/Location (secondary).
- `causal_mechanism`: prioritize Cause/Factor nodes (primary), then Effect/Outcome (primary), then Problem/Condition (secondary).
- `comparison`: prioritize Effect/Outcome nodes (primary), then Concept/Entity (secondary).
- `definition`: prioritize Concept/Entity nodes (primary), then Context/Location (secondary).

The compiler extracts the top-k nodes from each priority tier (typically 3–5 primary, 2–3 secondary) and includes them in the output. This ensures the compiled answer is type-aligned with the question intent.

Relation-to-Explanation Mapping. The compiler converts selected nodes and their connecting relations into natural-language explanations:

- `affects` → "affects", "impacts", "influences"
- `mitigates` → "mitigates", "reduces", "prevents"
- `caused_by` → "is caused by", "results from", "stems from"
- `leads_to` → "leads to", "produces", "results in"
- `studied_in` → "is studied in", "appears in", "is discussed in"
- `part_of` → "is part of", "is a component of"
- `depends_on` → "depends on", "requires"
- `correlates_with` → "correlates with", "is associated with"

For example, if the graph contains nodes "thermal stress" –affects–> "symbiodinium expulsion" –leads_to–> "coral bleaching", the compiler synthesizes: "Thermal stress affects symbiodinium expulsion, which leads to coral bleaching."

Structured Output Templates. The answer format varies by reasoning type:

- **Method/process**: numbered list of methods, each with: (1) method name, (2) prerequisites/dependencies, (3) expected outcome, (4) papers cited.
- **Comparison**: multi-row table with: rows = alternatives (Method nodes), columns = comparison dimensions (Effect/Outcome node names), entries = specific outcome values or trade-offs.
- **Causal mechanism**: linear narrative chain: "[Cause] occurs when [Context], leading to [Mechanism], which produces [Effect]". May branch if multiple mechanisms are present.

- **Definition**: primary definition (from Concept node + highest-degree edges), 2–3 related concepts (nearby Entity nodes), context in which definition varies.
- **Default**: bulleted list of key findings, each grounded in node + supporting edges.

Coverage Validation. Type-specific rules check whether the graph has sufficient evidence to answer the question confidently:

- Method questions require $\geq$3 Method/Intervention nodes. Fewer than 3 methods is incomplete for a "What methods exist?" question.
- Comparison questions require $\geq$2 distinct Concept/Entity or Effect/Outcome nodes. Cannot compare a single item to itself.
- Causal questions require $\geq$1 Cause/Factor + $\geq$1 Effect/Outcome + $\geq$1 reasoning path connecting them.
- Definition questions require $\geq$1 Concept/Entity node. Purely structural definitions require no secondary nodes, but prefer $\geq$1 related concept.

Coverage validation returns a tuple: (is_sufficient: bool, missing_requirements: List[str], coverage_score: float in [0, 1]).

If coverage is insufficient: The validation result is passed back to the adaptive retry loop (Stage 8). The missing requirements are encoded as targeted expansion dimensions:

- Missing "3 Method nodes" $\rightarrow$ generate expansion query: "What are alternative techniques, procedures, and approaches for [original query]?"
- Missing "Cause nodes" $\rightarrow$ generate expansion query: "What are the causes, drivers, and factors behind [original query]?"
- Missing "comparison data" $\rightarrow$ generate expansion query: "Compare and contrast [alternatives] with respect to [dimensions]".

Why a separate compiler instead of just using LLM output? The compiler provides three irreplaceable capabilities:

1. **Provenance and traceability**: every statement in the compiled answer traces back to specific graph nodes and edges, which trace back to specific papers and author names. Users can drill down to original evidence. An LLM's free-text answer has no such traceability; users must trust the model.
2. **Mechanically-verifiable coverage**: the compiler can check *automatically* whether the answer covers required entity types. This reliability is impossible for LLM self-assessment: models frequently claim to cover something they do not, or miss obvious gaps.
3. **Consistency and repeatability**: structured templates ensure answers follow the expected format and structure regardless of LLM variability. A method question always gets a numbered list; a comparison question always gets a table. This consistency is important for users building research workflows that depend on predictable answer structure.

3.12 Confidence Scoring and Adaptive Thresholds

The confidence scorer computes a final confidence score from multiple signals:

$$\text{confidence} = 0.35 \cdot \text{paper_support} + 0.30 \cdot \text{graph_density} + 0.20 \cdot \text{reasoning_paths} + 0.15 \cdot \text{node_diversity}$$

- **Paper support** (weight 0.35): normalized count of unique papers cited (scaled 0–1). The highest weight because multi-source corroboration is the strongest indicator of answer reliability.
- **Graph density** (weight 0.30): edges / (nodes × (nodes - 1)). Dense graphs indicate rich evidence interconnection and well-studied topics.
- **Reasoning paths** (weight 0.20): normalized count of extracted 2–4-hop paths. More paths = more ways to justify and understand the answer; more defensive against single-point-of-failure explanations.
- **Node diversity** (weight 0.15): Shannon entropy of entity type distribution. A graph with only "Concept" nodes is uniformly less informative than one with a mix of Causes, Methods, Effects, etc.

Interpretation and Actions. The confidence score guides decision-making throughout the pipeline:

- **>0.75 (High confidence):** Answer is well-supported. No adaptive expansion triggered. LLM generation proceeds with standard instructions.
- **0.55–0.75 (Medium confidence):** Answer is supported but not optimal. If adaptive expansion is enabled, a single targeted refinement may be run (Stage 8). LLM generation includes standard context without confidence caveats.
- <0.55 **(Low confidence):** Answer base is fragile. Triggers adaptive expansion (Stage 8) to retrieve additional papers and strengthen the graph. If expansion does not recover confidence, the low-confidence instruction is added to LLM context, cautioning the model to use hedging language.

Adaptive Thresholds. The 0.55 threshold is not universal; it adapts to query complexity:

- **Simple queries** (e.g., "Define X"): threshold raised to 0.65. Simple queries usually have lower density but that is acceptable.
- **Complex queries** (e.g., "What mechanisms contribute to X under Y conditions?"): threshold lowered to 0.50. Complex queries often have sparser evidence; we expand more eagerly rather than over-penalizing legitimate hard questions.
- **Multi-faceted queries** (e.g., "Compare A vs B vs C"): threshold raised to 0.70. These require diverse entity types; lower confidence is more concerning for completeness.

Why adaptive thresholds? A rigid threshold treats all queries equally, triggering expensive expansions on naturally-sparse-but-valid answers and wasting latency. Adaptive thresholds account for question structure, ensuring expansion happens where it adds value.

Coverage Signals and Early Stopping. The adaptive expansion loop (Stage 8) uses coverage validation (Stage 10's requirements) as a secondary stopping signal:

- If confidence is low *but* coverage is sufficient (has $\geq$3 Method nodes for a method question), expansion is skipped.
- If confidence is high *but* coverage is insufficient (e.g., Definition question missing Property nodes), expansion is triggered.
- This dual signal prevents both over-expansion on valid answers and under-expansion when the graph structure is fragmented.

Quality assessment maps the score to: `strong` / `moderate` / `weak` / `insufficient`, which is communicated to the user alongside the answer.

4 Cross-Cutting Design Decisions

4.1 Domain Agnosticity

The system works across Agriculture, Geoscience, Computer Science, Supply Chain, and Environment without any domain-specific code. This was a deliberate design choice:

- Entity types are abstract categories (Concept, Method, Cause, Effect...) that map to any domain.
- Relation types are generic scientific relationships (affects, causes, mitigates...) that apply universally.
- Query planning uses zero-shot LLM classification, not domain-specific rules.
- Domain detection is used *only* for retrieval source routing, not for constraining extraction or graph building.

The alternative would be domain-specific vocabularies (Gene, Protein, Pathway for biomedical; Layer, Formation, Basin for geoscience). This gives higher precision within a domain but requires per-domain engineering and breaks when queries span domains (e.g., "How does climate change affect agricultural supply chains?" spans Environment, Agriculture, and Supply Chain).

4.2 Session-Aware Caching and Incremental State Management

For conversational usage, the system maintains a **SessionState** object that persists artifacts across multiple queries within a user session. This enables dramatic latency reductions and coherent multi-turn interactions.

What is cached.

- **Detected domain**: once inferred, reused for all follow-up queries (avoids re-classifying on each turn).
- **Full knowledge graph**: the merged `nx.DiGraph` from prior queries.
- **Paper pool**: filtered and ranked papers from retrieval, stored by `paper_id` (up to 500 entries).

- **Extraction cache**: entity and relation tuples extracted from papers, keyed by paper ID + abstract hash (up to 300 entries).
- **Query embeddings and timestamps**: recent query texts, their embeddings, and timestamps.

Follow-up Query Similarity Decision. When a follow-up query arrives, its embedding is compared to the most recent query embedding via cosine similarity:

- **High similarity (>0.93)**: near-identical questions (e.g., "What is coral bleaching?" → "Tell me about coral bleaching"). Skip retrieval entirely, reuse cached papers and graph. Response latency: <1s. Confidence is inherited from the prior query.
- **Moderate similarity (0.82–0.93)**: related but distinct questions (e.g., "What causes coral bleaching?" → "How can coral bleaching be mitigated?"). Reuse the cached graph *and* retrieve additional papers targeting the new query intent. Graph is incrementally merged with new entities/relations. Response latency: ∼5s. Confidence is recomputed on the merged graph.
- **Low similarity (<0.82)**: substantially different queries (subject change, new domain, etc.). Full pipeline restart: clear the graph, run fresh retrieval, extraction, and graph building. Response latency: ∼15s.

Bounded Memory Management. Caches are bounded to prevent unbounded memory growth during long Streamlit sessions:

- Papers cache: max 500 entries. When exceeded, oldest entries (by insertion order) are evicted first (FIFO).
- Extraction cache: max 300 entries. Also evicted FIFO.
- Queries tracked: up to 20 recent query embeddings (for similarity matching).

TTL-Based Session Expiration. Sessions don't live forever. A session is considered stale if the time since the last query exceeds `SESSION_CACHE_TTL_SECONDS` (configurable; default: 3600 seconds = 1 hour). When a new query arrives after expiration:

- Caches are cleared, starting fresh.
- Query embeddings and timestamps are reset.
- The session is re-entered in a fresh state.

Why TTL expiration? Sessions can accumulate stale state. A user who pauses for an hour before asking a follow-up question likely expects independent answering, not reuse of stale cached graphs. The TTL prevents late-arriving follow-ups from using unrelated cached state.

Extraction Caching Within a Query. Even within a single query, extraction results are cached. If the adaptive expansion loop (Stage 8) adds new papers and re-extracts, the system checks the extraction cache first. If a paper's extraction was already computed (same abstract hash), it is reused rather than re-queried.

Why this reduces latency? Extraction is the most expensive stage (one LLM call per paper). In the typical case where adaptive expansion retrieves 5–10 new papers but a few overlap with the initial retrieval, skipping extraction on the overlapping papers saves ∼5–10 seconds.

Incremental Graph Merging. When new entities/relations are extracted in a follow-up or expansion, they are merged into the existing graph:

- Duplicate nodes are detected again via entity resolution.
- New edges are added, potentially creating bridges to existing components.
- The merged graph is revalidated (coverage check, confidence score updated).

This allows the graph to grow richer across a conversation rather than restarting from scratch.

4.3 Failure Handling Philosophy

The system handles failures **progressively** rather than aborting:

1. Retry with exponential backoff.
2. Re-prompt with strict JSON formatting.
3. Fall back to alternative model.
4. Fall back to deterministic heuristic extraction.
5. Compiler-driven partial structured output when coverage is insufficient.

Why progressive degradation? In a research tool, returning a partial answer is always better than returning an error. A user who asks "What methods exist for lithium extraction?" and gets a 60%-coverage answer with 3 methods cited is better served than a user who gets "Error: API rate limit exceeded". Every level of fallback trades quality for availability, and the confidence score communicates this trade-off honestly to the user.

4.4 Adaptive Answer Context Policy

The amount of graph context included in the LLM prompt is **not fixed**. It increases only when:

- Extraction success rate is above threshold (most papers yielded usable entities).
- The retrieved graph is validated as usable (meets minimum requirements).

Why? Sending a large, low-quality context to the LLM is worse than sending a small, high-quality one. If extraction mostly failed, the graph is noisy, and more context means more noise. The adaptive policy only "opens the tap" when the evidence quality justifies it.

5 Evaluation Framework

5.1 Benchmark Design

The evaluation harness includes 15+ benchmark questions across 5 domains, with both single-hop and multi-hop questions. Questions were designed to test different reasoning types (definition, comparison, causal, method, optimization).

5.2 Ablation Modes

To isolate the impact of each architectural addition, we evaluate four configurations:

1. **classic_rag**: top-3 paper abstracts concatenated as plain text. No graph, no extraction. This is the baseline—what a standard RAG system would produce.
2. **graphrag_base**: entity extraction + graph building, but no reasoning paths, no confidence-driven expansion, no query planning. Tests whether the graph layer alone adds value.
3. **graphrag_reasoning_only**: adds reasoning paths and structured retrieval, but no confidence-driven expansion. Tests whether explicit reasoning paths improve answers.
4. **graphrag_full**: all features enabled. Tests the complete system.

Why this ablation structure? Each mode adds one layer of complexity. Comparing adjacent modes isolates the contribution of that specific layer. If `graphrag_base` barely outperforms `classic_rag`, the graph layer alone is not enough. If `graphrag_full` significantly outperforms `graphrag_reasoning_only`, confidence-driven expansion is doing real work.

5.3 Metrics

- **ROUGE-1 F1**: unigram overlap—measures word-level content coverage.
- **ROUGE-2 F1**: bigram overlap—measures phrase-level content coverage.
- **Keyword Coverage**: percentage of reference answer keywords present in prediction—measures whether the answer addresses the key concepts.
- **Answer Length**: word count as a proxy for completeness.

6 Key Improvements Summary

Improvement	Purpose and Rationale
Query planning	Every downstream stage adapts to the question type. A method question retrieves, extracts, and formats differently from a causal question.
Entity resolution	Merges synonym/alias nodes to reduce graph fragmentation. Without this, "ML" and "machine learning" are separate disconnected nodes.
Hub-penalty retrieval	Prevents generic high-degree nodes from dominating the subgraph. Promotes specific, informative evidence chains.
Graph triples + paper abstracts in prompt	Combines structural reasoning (what connects to what) with narrative evidence (specific findings and numbers).
Semantic bridge edges	Connects entities across papers that never co-occurred but are semantically related. Enables cross-document reasoning paths.
Async concurrent extraction	Reduces extraction wall-time from ∼40s to ∼10s by parallelizing LLM calls (4 workers).

Model health precheck + fallback	Prevents hard failure when a configured model is unavailable or decommissioned.
Rate-limit graceful fallback	Returns degraded-but-useful output under 429/token-limit events instead of crashing.
Deterministic heuristic extraction fallback	When all LLM extraction calls fail, regex-based extraction preserves pipeline continuity.
Adaptive answer context policy	Expands LLM context only when extraction quality is high. Prevents noisy context from degrading answer quality.
Confidence-driven expansion loop	Detects insufficient evidence, diagnoses gaps, and re-retrieves targeted information. The key differentiator from static RAG.
Coverage validation per reasoning type	Mechanically checks whether the graph has the right entity types for the question type (e.g., method questions need Method nodes).
Answer compiler with structured templates	Produces formatted answers directly from graph evidence, with provenance tracing.
Session-aware caching	Follow-up queries reuse cached artifacts, reducing latency from $\sim$15s to <1s for near-identical queries.
Query expansion anchor validation	Prevents expansion drift by requiring $\geq$50% retention of original query terms.
Retrieval query token cap + dedup	Limits pathologically long search strings that caused upstream API timeouts.
Expansion-loop safeguards	Detects repeated/looping refinement queries and stops early to prevent retrieval drift.
Refinement query sanitization	Prevents missing-requirement labels from leaking into retrieval text.

7 Operational Behavior Under Different Conditions

- **Good API health, well-studied topic**: rich graph with 50+ nodes, multiple reasoning paths, high confidence (>0.7). Answer is structured per reasoning type with full citations. Typical latency: 10–15s.
- **Good API health, niche topic**: sparser graph, fewer papers. Adaptive expansion triggers 1–2 times. Confidence may be moderate (0.4–0.6). Answer is still structured but with fewer items. Latency: 20–35s (includes expansion).
- **Partial extraction failures**: some papers fail to extract. Confidence is lower, expansion may trigger. System falls back to heuristic extraction for failed papers. Returns output with honest confidence assessment.
- **Rate-limit pressure**: LLM calls return 429 errors. System uses backoff, then falls back to heuristic extraction. Answer quality degrades but system never crashes. Confidence score reflects the degradation.
- **Follow-up query in same session**: if query is similar (>0.93), cached graph is reused. Response in <1s. If moderately similar (0.82–0.93), graph is incrementally enriched. Response in $\sim$5s.

8 Component Map

File	Responsibility
`main.py`	Orchestration: runs all stages sequentially, manages confidence loop, adaptive top-k, metrics collection, session interactions.
`app.py`	Streamlit UI: end-to-end pipeline invocation, visualization, debug displays.
`pipeline/query_planner.py`	Reasoning-type classification, graph requirements, entity/dimension extraction, heuristic fallback.
`pipeline/domain_detector.py`	Three-tier domain classification (LLM → keyword → embedding).
`pipeline/paper_retriever.py`	Query expansion, dual retrieval, multi-source routing (Europe PMC, arXiv, OpenAlex), plan-aware scoring.
`pipeline/paper_filter.py`	Semantic ranking, adaptive percentile filtering, intent-aware boosting.
`pipeline/entity_extractor.py`	Parallel LLM extraction, retry cascade, heuristic fallback, extraction caching.
`pipeline/graph_builder.py`	Entity resolution, graph construction, generic node removal, co-occurrence/semantic bridge edges, isolated node pruning.
`pipeline/graph_retriever.py`	Seed selection, BFS expansion, hub penalty, reasoning path extraction, graph confidence scoring, subgraph validation.
`pipeline/answer_generator.py`	LLM answer generation with graph context, sparse graph supplementation, intent-driven length.
`pipeline/answer_compiler.py`	Graph-native answer compilation: node ranking, type-aware selection, structured templates, coverage validation.
`pipeline/confidence_scorer.py`	Multi-signal confidence scoring (paper support, density, paths, diversity), quality assessment.
`pipeline/adaptive_retry.py`	Gap analysis, targeted expansion queries, graph merging, bounded retry loop.
`pipeline/embedding_service.py`	Shared singleton encoder (all-MiniLM-L6-v2) + embedding cache.
`utils/session_state.py`	Session-level caches and query similarity reuse logic.
`evaluation/evaluator.py`	Benchmark harness, ablation modes, ROUGE/keyword metrics.

9 Tradeoffs and Known Constraints

- **Quality vs. cost**: richer graphs require more LLM calls (extraction, planning, expansion). Adaptive policies mitigate this (skip expansion when unnecessary) but do not eliminate the trade-off.
- **Dynamic vs. static**: per-query graph construction avoids staleness but incurs runtime overhead. For high-throughput deployment, a hybrid approach (cached base graph + query-specific overlay) could help.
- **Provider dependence**: the system relies on external LLM providers (Groq) and scholarly APIs (Europe PMC, arXiv). Rate limits and outages are handled gracefully, but sustained outages

degrade quality significantly.

- **Entity resolution precision**: aggressive merging risks collapsing distinct entities ("supervised learning" $\neq$ "unsupervised learning"). Conservative thresholds reduce this risk but leave some duplicates unmerged.
- **Expansion latency**: each retry iteration adds 10–20s. For latency-sensitive applications, the 2-iteration cap may need to be reduced to 1, trading answer completeness for speed.
- **Scholarly API variability**: upstream APIs (especially arXiv) can timeout or return empty results. Query dedup and token caps reduce this risk but cannot eliminate source-side instability.

10 Summary Narrative

We built a Dynamic GraphRAG system that transforms open scientific questions into structured, evidence-grounded answers. For each query, the system plans the reasoning strategy, retrieves fresh papers from multiple academic APIs, extracts entities and relations into a temporary knowledge graph, resolves duplicate entities, optimizes the graph structure, retrieves a focused subgraph with multi-hop reasoning paths, and compiles a structured answer with provenance.

The key differentiator is confidence-driven adaptive expansion: the system measures its own evidence quality, diagnoses what is missing, and re-retrieves targeted information—analogous to how a human researcher iterates on a literature search. Combined with progressive failure handling (the system never crashes, it degrades gracefully), session-aware caching (follow-up queries are near-instant), and reasoning-type-aware formatting (comparison questions get tables, causal questions get chains), the result is a research tool that produces better answers than flat RAG, explains where those answers come from, and honestly communicates its confidence.

11 Next Steps

- Offline regression suite with fixed snapshots for critical queries.
- Runtime metrics dashboard for confidence trends and bottleneck diagnosis.
- Per-domain threshold tuning using ablation results.
- Optional local-model fallback for provider outage scenarios.
- Expansion-loop guard and refinement-query sanitization regression tests.
- Per-stage time/token budget controls for predictable latency under heavy rate limiting.